Java Swing

Complete Concept Notes

JFrame . JLabel . JPanel

Source files covered:

- Frames.java (JFrame basics)
- Lable.java (JLabel basics)
- Panel.java (JPanel basics)

1. Introduction to Java Swing

Java Swing is a GUI (Graphical User Interface) toolkit that is part of the Java Foundation Classes (JFC). It is used to create window-based desktop applications such as calculators, forms, text editors, etc.

1.1 Key Facts

- Package used: javax.swing.*
- Swing is built on top of AWT (Abstract Window Toolkit), but unlike AWT, Swing components are lightweight (pure Java, not dependent on native OS widgets).
- Swing provides rich components: JFrame, JLabel, JPanel, JButton, JTextField, JCheckBox, etc.
- Swing follows the MVC (Model-View-Controller) design pattern internally.

1.2 Core Building Blocks Covered in These Notes

Method / Code	Explanation
`JFrame`	The main top-level window / container of a Swing application.
`JLabel`	A display area used to show text, an image, or both (non-editable).
`JPanel`	A lightweight container used to group / organize other components.

NOTE: A typical Swing application structure is: JFrame (the window) contains JPanel(s) (containers), which in turn contain components like JLabel, JButton, JTextField, etc.

2. JFrame - The Application Window

Definition: JFrame is a GUI window to which other components are added. It represents the main, top-level frame of a desktop application.

2.1 Basic Syntax

```
JFrame frame = new JFrame();
frame.setTitle("Calculator");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setResizable(false);
frame.setSize(420, 420);
frame.setVisible(true);
```

2.2 Commonly Used JFrame Methods

Method / Code	Explanation
`setTitle(String s)`	Sets the title text shown in the window's title bar.
`setDefaultCloseOperation(int op)`	Defines what happens when the user clicks the close (X) button. JFrame.EXIT_ON_CLOSE terminates the whole application.
`setResizable(boolean b)`	If false, prevents the user from resizing the frame.
`setSize(int w, int h)`	Sets the width and height (in pixels) of the frame.
`setVisible(boolean b)`	Makes the frame visible (true) or invisible (false) on screen.
`setIconImage(Image img)`	Changes the small icon shown in the title bar / taskbar, instead of the default Java (coffee-cup) logo.
`getContentPane().setBackground(Color c)`	Changes the background color of the frame's content area.
`setLayout(LayoutManager lm)`	Sets the layout manager. Passing null switches to absolute positioning (you must set bounds manually).
`pack()`	Sizes the frame to fit the preferred size of its child components.
`add(Component c)`	Adds a component (JLabel, JPanel, JButton, etc.) to the frame.

2.3 Setting a Custom Window Icon

By default, a Swing window shows the Java coffee-cup logo. This can be replaced using ImageIcon:

```
ImageIcon image = new ImageIcon("images.png"); // create an ImageIcon
frame.setIconImage(image.getImage());            // change the icon of the frame
```

2.4 Changing Background Color

```
frame.getContentPane().setBackground(Color.black);
```

```
// OR create a custom RGB color:
frame.getContentPane().setBackground(new Color(0, 0, 0));
```

2.5 Full Example (from Frames.java)

```
import javax.swing.JFrame;
import java.awt.Color;
import javax.swing.*;

class Frames {
    public static void main(String[] args) {

        JFrame frame = new JFrame();
        frame.setTitle("Calculator");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setResizable(false);
        frame.setSize(420, 420);
        frame.setVisible(true);

        ImageIcon image = new ImageIcon("images.png");
        frame.setIconImage(image.getImage());

        frame.getContentPane().setBackground(Color.black);
    }
}
```

NOTE: Common beginner mistake: calling setVisible(true) BEFORE adding all components. Ideally, configure the frame and add all components first, then call setVisible(true) last (or call revalidate()/repaint() if adding components after the frame is shown).

3. JLabel - Displaying Text and Images

Definition: JLabel is a GUI display area for a short string of text, an image, or both. It is a passive (non-interactive / non-editable) component, purely used for display purposes.

3.1 Basic Syntax

```
JLabel label = new JLabel();
label.setText("Bro, do you even code?");
```

3.2 Commonly Used JLabel Methods

Method / Code	Explanation
`setText(String s)`	Adds / changes the text displayed by the label.
`setIcon(Icon icon)`	Attaches an image (ImageIcon) to the label.
`setHorizontalTextPosition(int pos)`	Positions the TEXT horizontally relative to the icon: LEFT, CENTER, RIGHT.
`setVerticalTextPosition(int pos)`	Positions the TEXT vertically relative to the icon: TOP, CENTER, BOTTOM.
`setForeground(Color c)`	Sets the color of the text.
`setFont(Font f)`	Sets the font family, style (PLAIN/BOLD/ITALIC) and size of the text.
`setIconTextGap(int px)`	Sets the gap (in pixels) between the icon and the text.
`setBackground(Color c)`	Sets the background color of the label.
`setOpaque(boolean b)`	Must be set to true for setBackground() to actually be visible (labels are transparent by default).
`setBorder(Border b)`	Displays a border around the label (created via BorderFactory).
`setVerticalAlignment(int pos)`	Aligns the icon+text block vertically within the label's own bounds: TOP, CENTER, BOTTOM.
`setHorizontalAlignment(int pos)`	Aligns the icon+text block horizontally within the label's own bounds: LEFT, CENTER, RIGHT.
`setBounds(x, y, w, h)`	Manually sets position (x, y) and size (w, h) of the label. Only works when the parent's layout is set to null.

3.3 Creating a Border

```
Border border = BorderFactory.createLineBorder(Color.pink, 5);
label.setBorder(border);
```

BorderFactory.createLineBorder(Color color, int thickness) creates a simple solid-colored line border of the given thickness (in pixels).

3.4 Full Example (from Lable.java)

```
```

```
import java.awt.Color;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.border.Border;

class Lable {
    public static void main(String[] args) {

        ImageIcon image = new ImageIcon("one.jpg");
        Border border = BorderFactory.createLineBorder(Color.pink, 5);

        JLabel label = new JLabel();
        label.setText("Bro, do you even code?");
        label.setIcon(image);
        label.setHorizontalTextPosition(JLabel.CENTER);
        label.setVerticalTextPosition(JLabel.TOP);
        label.setForeground(Color.pink);
        label.setFont(new Font("MV Boil", Font.PLAIN, 50));
        label.setIconTextGap(100);
        label.setBackground(Color.black);
        label.setOpaque(true);
        label.setBorder(border);
        label.setVerticalAlignment(JLabel.CENTER);
        label.setHorizontalAlignment(JLabel.CENTER);

        JFrame frame = new JFrame();
        frame.setSize(700, 700);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setTitle("calculator");
        frame.add(label);
        frame.pack();
    }
}
```

3.5 Why pack() Matters

pack() automatically sizes the frame so that it fits the preferred size of the components added to it. If a component is added to the frame after setVisible(true) without calling pack() (or without the frame layout correctly configured), the component may not display properly. In the example above, pack() is called after add(label) to properly size the window around the label's content.

NOTE: setBounds() only has an effect when the container's layout manager is set to null (frame.setLayout(null)). Otherwise, the active LayoutManager (like FlowLayout, BorderLayout) decides the position and size automatically.

4. JPanel - Grouping Components

Definition: JPanel is a GUI component that functions as a container to hold other components (such as JLabel, JButton, JTextField, etc.). Panels help organize and group related UI elements, and can be nested inside a JFrame or inside other panels.

4.1 Basic Syntax

```
JPanel redpanel = new JPanel(); // a container that holds other components
```

4.2 Key Characteristics of JPanel

- JPanel has no visible border or title by default - it's an invisible grouping box.
- A JPanel can hold many child components and can itself be added to a JFrame or another JPanel.
- By default, JPanel uses FlowLayout as its layout manager.
- Panels are essential for building complex layouts: a frame can contain multiple panels, each with a different layout manager, for fine-grained control.

4.3 setLayout(null) - Absolute Positioning

```
frame.setLayout(null);
```

Setting the layout manager to null disables automatic layout management. This means every component added to that container MUST have its position and size manually defined using setBounds(x, y, width, height). This gives precise, pixel-level control (common in things like calculator UIs) but loses the automatic resizing / reflow benefits of layout managers.

4.4 Full Example (from Panel.java)

```
import javax.swing.JFrame;
import javax.swing.JPanel;

class Panel {
    public static void main(String[] args) {

        JPanel redpanel = new JPanel(); // container for other components

        JFrame frame = new JFrame();
        frame.setSize(700, 700);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(null);
```

```
        }
    }
```

NOTE: In this file the panel (redpanel) is created but never added to the frame via frame.add(redpanel) - so nothing will actually appear. To see the panel, you must call frame.add(redpanel) and, since the layout is null, also call redpanel.setBounds(x, y, w, h).

5. JFrame vs JLabel vs JPanel - Quick Comparison

Method / Code	Explanation
`JFrame`	The top-level window / container of the whole application. Only one (or a few) JFrame(s) typically exist per application.
`JPanel`	A mid-level container used to group / organize components. Added inside a JFrame (or another JPanel).
`JLabel`	A leaf-level display component used to show static text and/or images. Added inside a JFrame or JPanel.

5.1 Typical Component Hierarchy

```
JFrame (window)
   |
   +-- JPanel (container)
   |      |
   |      +-- JLabel (text/image)
   |      +-- JButton
   |      +-- JTextField
   |
   +-- JPanel (another section)
          +-- ...
```

5.2 Layout Manager vs setBounds / null layout

Method / Code	Explanation
`Default layout (e.g. FlowLayout)`	Components auto-positioned and resized by the layout manager. Simpler, more responsive, but less pixel-precise.
`setLayout(null) + setBounds(x, y, w, h)`	Manual / absolute positioning. Full pixel control, but does not resize or reflow with the window and is harder to maintain.

6. Quick Reference Cheat Sheet

6.1 Imports Used in These Files

```
import java.awt.Color;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.border.Border;
import javax.swing.*;
```

6.2 Minimal Reusable Template

```
import javax.swing.*;
import java.awt.*;

class MySwingApp {
    public static void main(String[] args) {

        JFrame frame = new JFrame();
        frame.setTitle("My App");
        frame.setSize(500, 500);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(null); // or leave default layout

        JPanel panel = new JPanel();
        panel.setBounds(20, 20, 400, 400);

        JLabel label = new JLabel("Hello Swing!");
        label.setBounds(20, 20, 200, 40);

        panel.add(label);
        frame.add(panel);

        frame.setVisible(true); // show the frame last
    }
}
```

6.3 Things to Remember / Common Pitfalls

- Call setVisible(true) after configuring the frame and adding components (or call revalidate()/repaint() afterward).
- setOpaque(true) is required for a JLabel's background color to actually show.
- setBounds() only works if the parent container's layout is set to null.
- pack() resizes the frame based on its components' preferred sizes - useful as an alternative to a fixed setSize().
- Always add a component to its parent container using add() - creating it alone is not enough for it to appear on screen.
- setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE) ensures the whole Java application exits when the window is closed (otherwise the JVM process may keep running in the background).

End of notes. Based on Frames.java, Lable.java and Panel.java from the JAVA_SWING project.